

Le langage C

François Trahay & Gaël Thomas



Contents

Présentation du module	2
Contenu du module	2
Evaluation du module	2
Déroulement d'une séance	3
Ressources disponibles	3
Conseils pour la suite de votre scolarité	4
Utilisation de ChatGPT (ou autre IA)	4
Utilisation des corrigés	4
Travail en groupe	5
C vs. Java	5
Mon premier programme en C	5
Déclaration de variable	6
Portée des variables	7
Opérateurs et Expressions	7
Opérateurs bit à bit	7
Remarque	8
Structures algorithmiques	9
Affichage / Lecture	9
Fonctions	9
Du type primitif au type composé	10
Les structures	10
Déclaration d'une variable de type structure	11
Accès aux champs d'une variable de type structure	12
Les tableaux	13
Accès aux éléments d'un tableau	13
Tableaux et structures	13
Différences par rapport à Java	14
Initialisation d'un tableau lors de sa déclaration	14

Initialisation mixte de tableaux et structures	15
Tableaux et chaînes de caractères	15
Typedef	15
Passage par valeur et par référence	16
Passage par valeur – les types primitifs	16
Passage par valeur – les structures	16
Passage par référence – les tableaux	17

Présentation du module

Objectifs du module:

- Maîtriser le langage C
- Savoir s’adresser au système d’exploitation depuis un programme

Modalités:

- Un peu de théorie
- Beaucoup de pratique

Contenu du module

Partie *Programmation*

- CI 1 – Le langage C
- CI 2 – Faire des programmes modulaires en C
 - Début du projet
- CI 3 – Les pointeurs
- CI 4 – Debugger un programme

Partie *Système*

- CI 5 – Les fichiers
- CI 6 – Les processus
- CI 7 – Appels système et Sémaphores
- CI 8 – Signaux

Evaluation

- CI 9 – Travail sur le projet
- CI 10 – Finalisation du projet et oral
- CF1 – Examen sur papier

Evaluation du module

Le module est évalué par

- un projet
 - but: que les étudiants programment
 - risque: certains étudiants peuvent “être aidés” (par un camarade/une IA)
 - évaluation par un oral/revue de code de 15 minutes.
 - * Exemple de questions:
 - *Comment as-tu implémenté la fonction `foo` ?*
 - *Que se passe-t-il si la fonction `bar(int n)` est appelée avec $n < 0$?*
 - un examen sur papier
 - but: évaluer la capacité des étudiants à résoudre un problème en C
-

Déroulement d’une séance

Système de *classe inversée*. Pour chaque séance :

- **Avant** la séance
 - Etude de la partie cours (y compris les commentaires des slides) de la séance à venir
- **Pendant** la séance:
 - Mini-évaluation de la partie cours (Kahoot!)
 - Explications sur les points mal compris
 - Travaux pratiques : expérimentations sur les concepts vus en cours

Attention ! Cela ne fonctionne que si vous travaillez sérieusement **avant** la séance.

Hypothèse: les étudiants suivant ce cours sont des adultes responsables.

Les supports de cours à étudier pour chaque séance sont disponibles dans plusieurs formats:

- Le Poly imprimé vous permet d’annoter le cours
 - Le poly est également disponible en pdf ou html
 - Pour chaque cours, les slides sont disponibles au format html. Appuyez sur la touche **S** pour afficher les notes de chaque slide.
-

Ressources disponibles

Pour vous aider, vous avez à votre disposition:

- Le poly contenant l’ensemble des transparents commentés
- Les transparents en version pdf/html
 - en html, appuyez sur **s** pour afficher les commentaires

- La documentation des fonctions C standard (`man 2 <fonction>` ou `man 3 <fonction>`)
 - Une équipe enseignante de choc !
-

Conseils pour la suite de votre scolarité

- N'utilisez pas d'IA pour vos TP/projets
 - ChatGPT / Copilot sont des outils pour **améliorer la productivité** des développeurs
 - Les TPs servent à **apprendre**, pas à produire
- Ne consultez les corrigés des TP qu'en dernier recours
 - Essayez de faire les exercices
 - Posez des questions aux enseignants

Utilisation de ChatGPT (ou autre IA)

Les IA génératives peuvent améliorer la productivité des développeurs. Il existe des plugins qui s'intègrent directement dans certains IDE et qui permettent de générer du code à la place du développeur. Si ces outils peuvent être utiles pour un développeur dans une entreprise, ils sont à proscrire dans le cadre de vos études. Outre les problèmes éthiques et écologiques, ces outils peuvent être néfastes lorsqu'il s'agit d'apprendre à programmer.

Le but des modules que vous suivez est de vous apprendre à programmer et à comprendre certains concepts ou problèmes. Si une IA écrit le code à votre place, vous ne pouvez pas être confrontés à ces problèmes/concepts et vous n'êtes pas capable de vérifier que le code généré est correct.

Par ailleurs, lorsqu'on vous évalue (que ce soit lors d'un projet, ou TP noté), vos enseignants évaluent votre travail, pas celui d'une IA. Si vous rendez du code généré par une IA, il est possible que d'autres étudiants fassent de même, et la probabilité pour que vous rendiez le même code (ou du code similaire) est forte. Ceci s'apparente à la triche, ce qui est fortement sanctionné à Télécom SudParis (0/20 au module, impossibilité de passer un éventuel CF2, potentielles sanctions supplémentaires décidées par la DF)

Il convient donc de désactiver ces éventuels plugins dans vos IDE, que ce soit pendant les TP, ou les projets. Vous êtes bien sûr libres de les réactiver pour vos projets perso.

Utilisation des corrigés

Les corrigés des différents TP sont disponibles en ligne et dans certains modules, des enregistrements vidéo des TP viennent compléter les ressources que nous proposons aux étudiants pour les aider. Il arrive que des étudiants se contentent de copier-coller les corrigés, puis compiler et tester, avant de passer à l'exercice

suivant. Ces étudiants ne sont donc pas confrontés aux problèmes qu'on souhaite montrer et n'apprennent pas à les résoudre par eux-même.

Il convient donc de faire les TP par vous même. Si vous êtes bloqués, posez des questions à l'enseignant qui vous mettra sur la voie. En faisant cela, vous progresserez et serez capable de faire les TP sans avoir le corrigé (ce qui sera utile lors de l'examen final, mais également dans votre vie professionnelle future).

Travail en groupe

L'entraide est bien sûr encouragée dans nos enseignements. Lorsque vous ne comprenez pas un concept ou lorsque vous révisez, n'hésitez pas solliciter vos camarades. Par contre, lorsqu'il s'agit d'un travail évalué (projet, DM, TP noté, etc.), cette entraide est contre-productive. Notre but est d'évaluer vos compétences et vous faire progresser. Récupérer le code d'un camarade, ou partager votre code avec eux (que ce soit en s'envoyant des fichiers, via un repo git, ou via une messagerie type discord) est très risqué. En cas de triche, tous les tricheurs (qu'ils aient récupéré le code d'un camarade, ou partagé le leur) auront 0/20 au module.

D'une manière générale, vous avez tous un bon niveau en informatique et vous devriez tous pouvoir valider ce module si vous y travaillez. Nous sommes conscients du fait que votre emploi du temps est parfois très chargé. Mais vous pouvez le faire par vous même sans tricher.

C vs. Java

- langage de *bas niveau* vs. *haut niveau*
- En C, manipulation de la mémoire et de ressources proches du matériel
- “*Un grand pouvoir implique de grandes responsabilités*”¹
- programmation impérative vs. programmation objet

¹ B. Parker, *Amazing Fantasy*, 1962

Mon premier programme en C

Fichier *.c

```
/* hello_world.c */
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char** argv) {
    printf("Hello World!\n");
}
```

```
    return EXIT_SUCCESS;
}
```

Compilation/execution:

```
$ gcc hello_world.c -o hello_world -Wall -Werror
$ ./hello_world
Hello World!
```

- Les `#include <stdio.h>` indiquent que le programme a besoin des outils `stdio`. Il s'agit donc d'un équivalent du `import package` de Java
- Pour afficher un message dans le terminal, on utilise la fonction `printf("message\n");`
- Le `return EXIT_SUCCESS;` à la fin du `main` permet de spécifier le code retour du programme (accessible depuis le shell en faisant `echo $?`). En cas d'erreur, on peut retourner `EXIT_FAILURE` à la place de `EXIT_SUCCESS`.

Déclaration de variable

Pour les types simples, déclaration identique à Java:

```
int var1;
int var2, var3, var4;
int var5 = 42;
```

Types disponibles:

- pour les entiers: `int`, `short`, `long`, `long long`
- pour les flottants: `float`, `double`
- pour les caractères: `char`

Pour les entiers: possibilité de préfixer le type par `unsigned`. Les variables sont alors non-signées (ie. positives).

La taille d'une variable entière (ie. le nombre de bits/octets) dépend de l'implémentation. Le standard C ne spécifie que la taille minimum. Ainsi, un `int` doit faire au moins 16 bits, alors que la plupart des implémentations modernes utilisent 32 bits pour les `int`. Il convient donc de ne pas se reposer sur ces types lorsqu'on a besoin d'un nombre précis de bits/octets.

Pour cela, il est préférable d'utiliser les types fournis par `stdint.h`: `uint8_t` (8 bits), `uint16_t` (16 bits), `uint32_t` (32 bits), ou `uint64_t` (64 bits).

Comme en Java, les variables déclarées dans une fonction sont *locales* à la fonction (elles disparaissent donc dès la sortie de la fonction). Les variables déclarées en dehors d'une fonction sont *globales*: elles sont accessibles depuis n'importe quelle fonction.

Portée des variables

Une variable est accessible dans le *bloc* dans laquelle elle est déclarée

```
void foo() {
    if(x==0) {
        int a = 17;
        for(int i=0; i< 10; i++) {
            int a = i;
            printf("  a=%d\n", a); // affiche a=0, a=1, ... a=9
        }
        printf("a=%d\n", a); // affiche a=17
    }
    printf("a=%d\n", a); // erreur de compilation: a n'existe pas
}
```

Une **variable globale** est déclarée en dehors de tout bloc. Elle est accessible depuis tout le fichier.

Opérateurs et Expressions

La liste des opérateurs disponibles est à peu près la même qu'en Java:

- arithmétique : +, -, *, /, %
- affectation : =, +=, -=, *=, /=, %=
- incrémentation/décrémentation: ++, --
- comparaison: <, <=, >, >=, ==, !=
- logique: !, &&, ||

Mais également:

- `sizeof n`: donne le nombre d'octets qui constitue une variable/un type `n`
-

Opérateurs bit à bit

Possibilité de travailler sur des *champs de bits*.

- Opération sur les bits d'une variable
- décalage: <<, >>
- OR : |, AND :&, XOR : ^, NOT : ~
- affectation: <<=, >>=, |=, &=, ^=, ~=

```
/* bits.h */
```

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
```

```

int main(int argc, char** argv) {
    uint32_t v = 1;
    int i;

    /* l'opérateur << décale vers la gauche */
    for(i=0; i<32; i++) {
        /* v << i décale les bits de v de i places vers la gauche
         * c'est équivalent à calculer v*(2^i)
         */
        printf("v<<%d = %u\n", i, v<<i);
    }

    v = 5;
    /* v | 3 effectue un OU logique entre les bits de v et la représentation binaire de 3
     * 101 | 11 = 111 (7)
     */
    printf("%u | %u = %u\n", v, 3, v|3);

    /* v & 3 effectue un ET logique entre les bits de v et la représentation binaire de 3
     * 101 & 11 = 001 (1)
     */
    printf("%u & %u = %u\n", v, 3, v&3);

    /* v ^ 3 effectue un XOR logique entre les bits de v et la représentation binaire de 3
     * 101 ^ 011 = 110 (6)
     */
    printf("%u ^ %u = %u\n", v, 3, v^3);

    /* ~v effectue un NON logique des bits de v
     * ~ 00...00101 = 11..11010 (4294967290)
     */
    printf("~%u = %u\n", v, ~v);

    return EXIT_SUCCESS;
}

```

Remarque

Lorsqu'on opère un décalage (avec <<) sur une valeur signée (par exemple, un `int`), le bit de signe n'est pas modifié par le décalage. Par exemple, si les bits d'un `int a` sont à 1010 0000 0000 0000 0000 0000 0000, le résultat de `a >> 1` est 1001 0000 0000 0000 0000 0000 0000.

Structures algorithmiques

Comme en Java:

- `for(i=0; i<n; i++) { ... }`
 - `while(cond) { ... }`
 - `do { ... } while(cond);`
 - `if (cond) { ... } else { ... }`
 - `break` pour sortir d'une boucle
-

Affichage / Lecture

- Pour afficher: `printf("%d exemple de %f format \n", v1, v2);`
- Pour lire: `scanf("%d-%f", &v1, &v2);`

```
/* formats.c */
#include <stdio.h>

int main(int argc, char** argv) {
    int v;
    printf("Entrez la valeur de v:\n");
    scanf("%d", &v);
    printf("v = %d (en decimal)\n", v);
    printf("v = %u (en decimal non signe)\n", v);
    printf("v = %x (en hexadecimal)\n", v);
    printf("v = %o (en octal)\n", v);
    printf("v = %c (en ASCII)\n", v);

    double a;
    scanf("%lf", &a);
    printf("a = %f (en flottant)\n", a);
    printf("a = %lf (en flottant double precision)\n", a);
    printf("a = %e (en notation scientifique)\n", a);

    char *chaine = "Bonjour";
    printf("chaine = %s\n", chaine);
    printf("chaine = %p (adresse)\n", chaine);

    printf("On peut aussi afficher le caractère %%\n");
}
```

Fonctions

Déclaration:

```

type_retour nom_fonc(type_param1 param1, type_param2 param2) {
    /* déclaration des variables locales */
    /* instructions à exécuter */
}

```

En C, il est d'usage de nommer les fonctions en minuscule, en séparant les mots par `_`. Cette convention se nomme snake case.

Par exemple, l'équivalent en C de la fonction Java `calculerLeMinimum()` (qui utilise la convention de nommage camel case) sera `calculer_le_minimum()`.

Du type primitif au type composé

- Jusqu'à maintenant, vous avez vu les types primitifs
 - `char`, `short`, `int`, `long` `long` (signés par défaut, non signés si préfixés de `unsigned`)
 - `float` (4 octets), `double` (8 octets)
 - Dans ce cours, nous apprenons à définir de nouveaux types de données
 - Une **structure** est constituée de sous-types **hétérogènes**
 - Un **tableau** est constitué de sous-types **homogènes**
-

Les structures

Une structure est une définition d'un nouveau type de données

- composé de sous-types nommés (primitifs ou composés)
- possédant un nom

Remarque: les sous-types d'une structure s'appellent des champs

Définition d'une nouvelle structure avec :

```

struct nom_de_la_structure {
    type1 nom_du_champs1;
    type2 nom_du_champs2;
    ...
};

```

Par exemple:

```

struct nombre_complexe {
    int partie_reelle;
    int partie_imaginaire;
};

```

Par convention, les noms de structures commencent par une minuscule en C

Une structure peut aussi être composée à partir d'une autre structure, comme dans cet exemple:

```
struct point {
    int x;
    int y;
};

struct segment {
    struct point p1;
    struct point p2;
};
```

En revanche, une structure ne peut pas être composée à partir d'elle-même. À titre d'illustration, l'exemple suivant n'est pas correct:

```
struct personnage {
    struct personnage ami;
    int point_de_vie;
};
```

Cette construction est impossible car il faudrait connaître la taille de la structure `personnage` pour trouver la taille de la structure `personnage`.

Déclaration d'une variable de type structure

- Une déclaration d'une variable de type structure se fait comme avec un type primitif:

```
struct nombre_complexe z1, z2, z3;
```

- On peut aussi initialiser les champs de la structure lors de la déclaration:

```
/* partie_relle de z prend la valeur 0 */
/* partie_imaginaire de z prend la valeur 1 */
struct nombre_complexe i = { 0, 1 };
/* autre solution : */
struct nombre_complexe j = { .partie_reelle=0, .partie_imaginaire=1 };
```

L'initialisation d'une variable de type structure est différente lorsque la variable est déclarée globalement ou localement. On vous rappelle qu'une variable globale si elle est déclarée en dehors de toute fonction. Sinon, on dit qu'elle est locale.

Lorsqu'une variable de type structure est déclarée en tant que variable globale sans être initialisée, le compilateur initialise chacun de ces champs à la valeur 0. En revanche, lorsqu'une structure est déclarée en tant que variable locale dans une fonction sans être initialisée, ces champs prennent une valeur aléatoire.

Par exemple, dans :

```
struct nombre_complexe i;
```

```
void f() {  
    struct nombre_complexe j;  
}
```

Les champs de `i` sont initialisés à 0 alors que ceux de `j` prennent une valeur aléatoire.

On peut aussi partiellement initialiser une structure comme dans l'exemple suivant :

```
struct nombre_complexe j = { 1 };
```

Dans ce cas, le champs `partie_relle` prend la valeur 1 et le champs `partie_imaginaire` prend soit la valeur 0 si la variable est globale, soit une valeur aléatoire si la variable est locale à une fonction.

Accès aux champs d'une variable de type structure

- L'accès aux champs d'une variable de type structure se fait en donnant le nom de la variable, suivi d'un point, suivi du nom du champs :

```
struct point {  
    int x;  
    int y;  
};  
  
struct ligne {  
    struct point p1;  
    struct point p2;  
};  
  
void f() {  
    struct point p;  
    struct ligne l;  
  
    p.x = 42;  
    p.y = 17;  
    l.p1.x = 1;  
    l.p1.y = 2;  
    l.p2 = p; /* copie p.x/p.y dans l.p2.x/l.p2.y */  
  
    printf("[%d %d]\n", p.x, p.y);  
}
```

Les tableaux

- Un tableau est un type de données composé de sous-types homogènes
 - Les éléments d'un tableau peuvent être de n'importe quel type (primitif, structure, mais aussi tableau)
 - Pour déclarer un tableau:

`type_des_elements nom_de_la_variable[taille_du_tableau];`

Par exemple:

```
int      a[5];      /* tableau de 5 entiers */
double   b[12];     /* tableau de 12 nombres flottants */
struct point c[10];  /* tableau de 10 structures points */
int      d[12][10]; /* tableau de 10 tableaux de 12 entiers */
                        /* => d est une matrice 12x10 */
```

Accès aux éléments d'un tableau

- L'accès à l'élément `n` du tableau `tab` se fait avec `tab[n]`
- Un tableau est indexé à partir de zéro (éléments vont de 0 à N - 1)

```
void f() {
    int x[3];
    int y[3];
    int i;

    /* 0 est le premier élément, 2 est le dernier */
    for(i=0; i<3; i++) {
        x[i] = i;
        y[i] = x[i] * 2;
    }
}
```

Tableaux et structures

- On peut mixer les tableaux et les structures, par exemple :

```
struct point {
    int x;
    int y;
};

struct triangle {
    struct point sommets[3];
};
```

```

void f() {
    struct triangle t;

    for(i=0; i<3; i++) {
        t.sommets[i].x = i;
        t.sommets[i].y = i * 2;
    }
}

```

Différences par rapport à Java

- On ne peut pas accéder à la taille d'un tableau
- Lors d'un accès en dehors des bornes du tableau, l'erreur est silencieuse :
c'est une erreur, mais elle n'est pas signalée immédiatement
=> parmi les erreurs les plus fréquentes (et les plus difficiles à repérer) en C

```

void f() {
    int x[3];

    x[4] = 42; /* Erreur silencieuse !!! */
               /* Écriture à un emplacement aléatoire en mémoire */
               /* le bug pourra apparaître n'importe quand */
}

```

Initialisation d'un tableau lors de sa déclaration

- Un tableau peut être initialisé lorsqu'il est déclaré avec

```
type_element nom_variable[taille] = { e0, e1, e2, ... };
```

- Par exemple: `int x[6] = { 1, 2, 3, 4, 5, 6 };`
- Comme pour les structures, on peut partiellement initialiser un tableau
 - Par exemple: `int x[6] = { 1, 1, 1 };`

En l'absence d'initialisation :

- Si le tableau est une variable globale, chaque élément est initialisé à 0
- Sinon, chaque élément est initialisé à une valeur aléatoire

Lorsqu'on initialise un tableau lors de sa déclaration, on peut omettre la taille du tableau. Dans ce cas, la taille du tableau est donnée par la taille de la liste d'initialisation.

```
int x[] = { 1, 2, 3 }; /* tableau à trois éléments */  
int y[6] = { 1, 2, 3 }; /* tableau à six éléments, avec les trois premiers initialisés */
```

Initialisation mixte de tableaux et structures

- On peut composer des initialisations de tableaux et de structures

```
struct point {  
    int x;  
    int y;  
};  
  
struct triangle {  
    struct point sommets[3];  
};  
  
struct triangle t = {  
    { 1, 1 },  
    { 2, 3 },  
    { 4, 9 }  
};
```

Tableaux et chaînes de caractères

Une chaîne de caractère est simplement un tableau de caractères terminé par le caractère '\0' (c'est à dire le nombre zéro)

```
char yes[] = "yes";
```

est équivalent à

```
char yes[] = { 'y', 'e', 's', '\0' };
```

Typedef

L'instruction `typedef` permet de donner un nom à type:

```
typedef int entier;  
entier i=5; // équivalent à int i=5;
```

```
typedef struct node node_t;  
node_t n; // déclare un struct node
```

```
typedef char[16] name;
```

```
// définit à la fois le type 'struct complexe' et le type 'complexe'
typedef struct complexe { int partie_reelle; int partie_imaginaire } complexe_t;
```

Passage par valeur et par référence

- En C, il existe deux types de passage d'arguments :
 - Passage **par valeur** : l'argument est copiée de l'appelé vers l'appelant
=> l'argument et sa copie sont deux variables différentes
 - Passage **par référence** : une référence vers l'argument de l'appelant est donné à l'appelé
=> l'appelant et l'appelé partagent la même donnée
 - Par défaut :
 - Les **tableaux** sont passés par **référence**
 - * Un argument de type tableau est déclaré avec `type nom[]`, sans la taille
 - Les **autres types** sont passés par **valeur**
-

Passage par valeur – les types primitifs

```
/* le x de f et le x du main sont deux variables distinctes */
/* le fait qu'elles aient le même nom est anecdotique */
void f(int x) {
    x = 666;
    printf("f : x = %d\n", x);           /* f : x = 666 */
}

int main() {
    int x = 42;
    f(x);                               /* x est copié dans f */
    /* => le x de main n'est donc pas modifié par f */
    printf("g : x = %d\n", x);           /* g : x = 42 */
    return 0;
}
```

Passage par valeur – les structures

```
struct point {
    int x;
    int y;
};
```

```

void f(struct point p) {
    p.x = 1;
    printf("(%d, %d)\n", p.x, p.y);      /* => (1, 2) */
}

int main() {
    struct point p = { -2, 2 };
    f(p);                                /* p est copié dans f */
    printf("(%d, %d)\n", p.x, p.y);      /* => (-2, 2) */
    return 0;
}

```

Passage par référence – les tableaux

```

void print(int x[], int n) {
    for(int i=0; i<n; i++) {
        printf("%d ", x[i]);
    }
    printf("\n");
}

int main() {
    int tab[] = { 1, 2, 3 };

    print(tab, 3); /* => 1 2 3 */
    f(tab);
    print(tab, 3); /* => 1 42 3 */

    return 0;
}

/* x est une référence vers le tableau original */
void f(int x[]) {
    x[1] = 42;      /* => modifie l'original */
}

```
